
CT SCAN RECONSTRUCTION WITH DEEP LEARNING

Luuk Fröling

1st Jan, 2024

created in  Curvenote

1 Generating a phantom and a sinogram

- Generate a phantom where each pixel consists of part bone and part water described as (bone, water)
 - Background values should be low for both to simulate air (0, 0.05)
 - main body should be elliptical and mainly consist of water (0.20, 0.80)
 - elliptical inserts should simulate different organs and/or body features (0.80, 0.80)
- Generate a sinogram from the projections
 - The function will return a 3D object containing the sinograms for the height of the detector. We want to select a slice to display
 - print a 2D cross section of the phantom with a corresponding sinogram

```
#settings
objectSize = 32      # size of the object
nPixelsY = 64        # number of pixels in Y direction detector
nPixelsZ = 44        # number of pixels in Z direction detector
projections = 64     # number of projections
```

1.0.1 generate phantom

```
import random, numpy as np
```

```
def generate_ct_matrices(n, h, num_features=5):
```

```
    # Initialize 3D bone and water matrices with zeros
```

```
    bone_matrix = np.zeros((n, n, n), dtype=float)
```

```
    water_matrix = np.zeros((n, n, n), dtype=float)
```

```
    # fill background of matrices with 0.05 bone and water
```

```
    bone_matrix.fill(0)
```

```
    water_matrix.fill(0.05)
```

```
    # set values for main ellipse (simulated torso)
```

```
    intensity_bone = 0.2
```

```
    intensity_water = 0.8
```

```
    # Main body ellipse (simulated torso)
```

```
    for i in range(h, n - h):
```

```
        for j in range(2 * h, n - 2 * h):
```

```
            if ((i - n//2) / ((n - 2 * h) / 2)) ** 2 + ((j - n//2) / ((n - 6 * h) / 2)) ** 2 <= 1:
```

```
                bone_matrix[2*h: -2*h, i, j] = intensity_bone
```

```
                water_matrix[2*h: -2*h, i, j] = intensity_water
```

```
    # Adding smaller features (e.g., organs or bone structures)
```

```
    for _ in range(num_features):
```

```
        # Random place in the body - h is the offset from the edge, n is the size of the matrix
```

```
        a = random.randint(h, (n - h) // 4) # Semi -major axis
```

```
        b = random.randint(h, (n - h) // 4) # Semi -minor axis
```

```
        center_x = random.randint(h + a, n - h - a)
```

```
        center_y = random.randint(3 * h + b, n - 3 * h - b)
```

```
        # Randomly choose the intensity for bone and water, together they add to 1.
```

```

bone_intensity = 0.8
water_intensity = 0.8

for i in range(center_x - a, center_x + a):
    for j in range(center_y - b, center_y + b):
        if 0 <= i < n and 0 <= j < n:
            if ((i - center_x) / a) ** 2 + ((j - center_y) / b) ** 2 <= 1:
                bone_matrix[2*h: -2*h, i, j] = bone_intensity
                water_matrix[2*h: -2*h, i, j] = water_intensity
water_matrix *= 1 # Water density is 1
bone_matrix *= 1.92 # Apply bone density

return bone_matrix, water_matrix

bone, water = generate_ct_matrices(objectSize, 2, num_features=3)
x = np.column_stack((bone.ravel(), water.ravel()))

```

1.0.2 Running projection algorithm

```

from libs.simulatepreps import projectMatrix

#generate proection matrices
y, S, M, A = projectMatrix(x, objectSize, nPixelsY, nPixelsZ, 1, projections)

```

1.0.3 Generating sinogram

```

def getSinogram(y, energy_bin_index=0, z_index=32, nProj=100, detector_shape=(64, 44)):
    nX, nZ = detector_shape
    y_bin = y[energy_bin_index]
    y_proj_zx = y_bin.reshape((nProj, nZ, nX)) # shape: (100, 64, 44)
    sinogram = y_proj_zx[:, z_index, :] # shape: (100, 44)
    return sinogram

```

```
sinogram = getSinogram(y, energy_bin_index=0, z_index=objectSize // 2, nProj=projections, detector_shape=(64, 44))
```

1.0.4 Make figure

```

import matplotlib.pyplot as plt

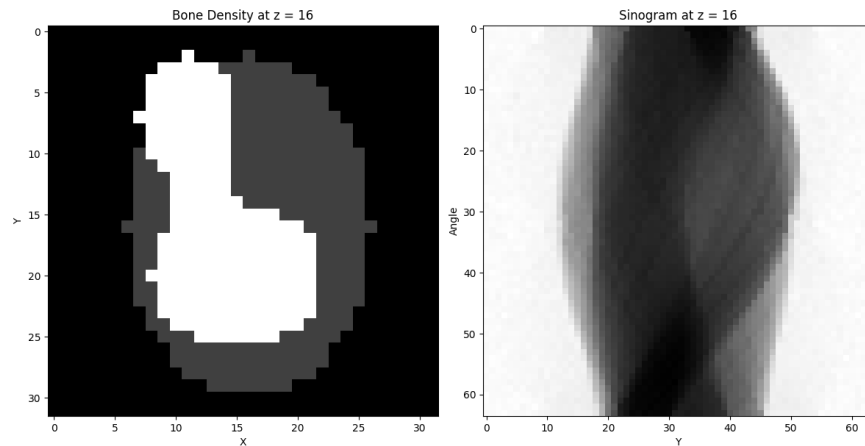
# plot bone density and phantom at objectSize //2
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(bone[objectSize // 2, :, :], cmap='gray')
plt.title('Bone Density at z = {}'.format(objectSize // 2))
plt.xlabel('X')
plt.ylabel('Y')

plt.subplot(1, 2, 2)
plt.imshow(sinogram, cmap='gray')
plt.title('Sinogram at z = {}'.format(objectSize // 2))
plt.ylabel('Angle')
plt.xlabel('Y')

plt.tight_layout()

```

```
plt.show()
```



1.1 Visualising a 2D convolution

```
# we want to put the phantom through a pytorch 2d convolution layer
import torch
```

```
bone_array = bone[objectSize // 2, :, :]
bone_tensor = torch.tensor(bone[objectSize // 2, :, :], dtype=torch.float32)
```

```
# create a 2D convolution layer with a kernel size of 3x3
conv_layer = torch.nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1)
```

```
# apply the convolution layer to the bone tensor
output_tensor = conv_layer(bone_tensor.unsqueeze(0).unsqueeze(0)) # Add batch and channel dimensions
output_image = output_tensor.squeeze(0).squeeze(0).detach().numpy() # Remove batch and channel dimensions
```

```
# plot 2 plots, one of the original bone image with a square showing one of the convolution kernels, and
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(bone_tensor.numpy(), cmap='gray')
# draw a square representing the convolution kernel
kernel_size = conv_layer.kernel_size[0]
plt.gca().add_patch(plt.Rectangle((10.5, 20.5), kernel_size, kernel_size, edgecolor='red', facecolor='none'))
# fill square with rectangles for each pixel
for i in range(kernel_size):
    for j in range(kernel_size):
        plt.gca().add_patch(plt.Rectangle((10.5 + i, 20.5 + j), 1, 1, edgecolor='red', facecolor='none'))
plt.title('Original Bone Image at z = {}'.format(objectSize // 2))
plt.xlabel('X')
plt.ylabel('Y')
plt.subplot(1, 2, 2)
plt.imshow(output_image, cmap='gray')
# draw little square representing the pixel on which the convolution was applied
plt.gca().add_patch(plt.Rectangle((10.5+2, 20.5+2), 1, 1, edgecolor='red', facecolor='none', lw=2))
plt.title('Output Image after Convolution')
plt.xlabel('X')
plt.ylabel('Y')
```

Text(0, 0.5, 'Y')



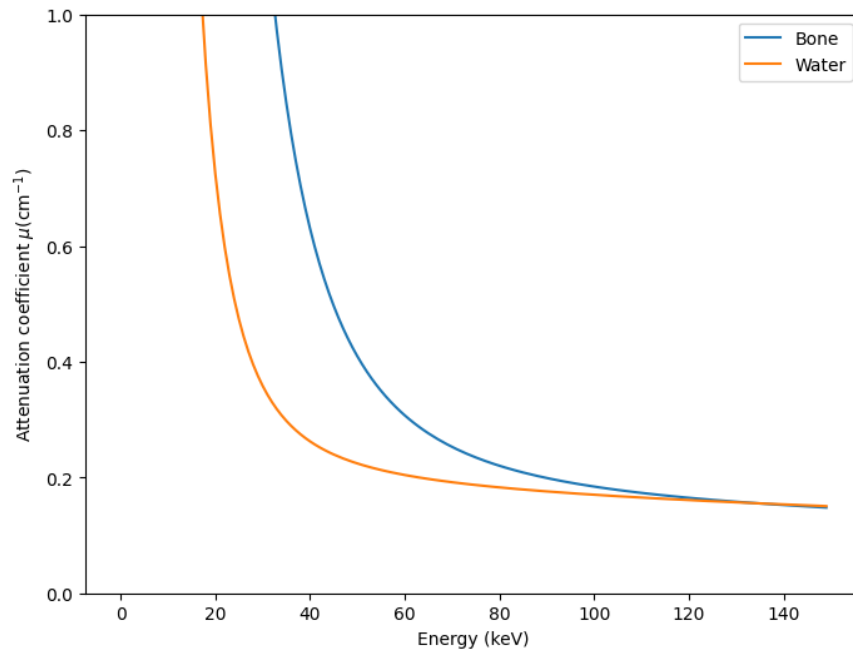
2 Load attenuation coefficients

The attenuation coefficients change depending on the energy of the photons used. Load values and plot these.

```
import scipy.io
import matplotlib.pyplot as plt

# Load the .mat file
mat_data = scipy.io.loadmat('libs/materialAttenuationBoneWater.mat')

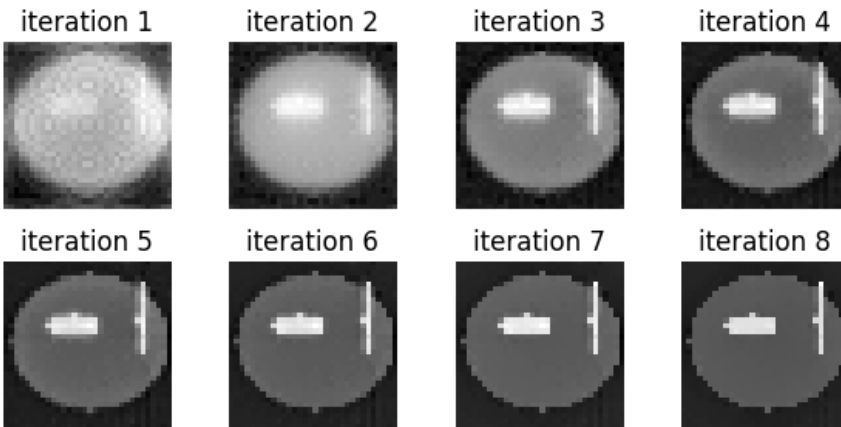
first = mat_data['materialAttenuations'][:,0]
second = mat_data['materialAttenuations'][:,1]
plt.figure(figsize=(8, 6))
plt.plot(first, label='Bone')
plt.plot(second, label='Water')
plt.xlabel('Energy (keV)')
#nicely format the unit
plt.ylabel(r"Attenuation coefficient  $\mu(\text{cm}^{-1})$ ")
plt.legend()
plt.ylim(0,1)
plt.show()
```



```
import pickle
import matplotlib.pyplot as plt

#load iteration.pkl
with open('iteration.pkl', 'rb') as f:
    x = pickle.load(f)

# make 2x4 plot with all images in x
plt.figure(figsize=(6, 3))
for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(x[i][0, :, 8:40, 16], cmap='gray') # e.g., bone input
    plt.title(f"iteration {i+1}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



3 Structuring the data

- append image to sinogram
- fix dimension issues using 0 padding
- fix multiple of 16 issues using 0 padding

As input data we will append the image to the sinogram. There will be a mismatch in height which we will solve by using 0 padding. Due to the convolution layers in the network, it is desired for the input dimensions to be multiples of 16.

```
import numpy as np
from libs.simulatepreps import projectMatrix
```

3.1 Define phantom and generate projection matrix

```
# generate a bone and water phantom with a 32x32 size
## SETTINGS ##
objectSize = 32                                # Object Size a square
nPixelsY = 64                                  # Number of pixels on the detector plate.
nPixelsZ = 44                                  # Number of pixels on the detector plate.
pixelPitch = 1                                 # Pixel pitch
nPixelsPerProj = nPixelsZ * nPixelsY           # The total number of pixels on the detector plate
projections = 32                                # Number of projections
```

```
def generatePhantom(size):
    bone_cylinder = np.full((size, size, size), 0.05, dtype=np.float32)
    water_cylinder = np.full((size, size, size), 0.05, dtype=np.float32)

    radius = 0.3 * size
    center = size // 2

    for x in range(size):
        for y in range(size):
            for z in range(size):
                if ((x - center) ** 2 + (y - center) ** 2) <= radius ** 2:
                    if (z > 2 and z < size - 2):
                        bone_cylinder[x, y, z] = 1.0
                        water_cylinder[x, y, z] = 1.0

    return bone_cylinder, water_cylinder

bone, water = generatePhantom(objectSize)
x = np.column_stack((bone.ravel(), water.ravel()))
y, _, _, _ = projectMatrix(x, objectSize, nPixelsY, nPixelsZ, pixelPitch, projections)
```

3.2 define prepareInput and prepareOutput

```
def prepareInput(y):

    # Curriculum learning input preparation function
    # As an input: 10 channels consisting of empty images next to the sinograms
```



```

# shape: 32x48x96, beginning with a 32x32x32 zero image with padding and a 32x44x64 image with padding

zeroImage = np.zeros((objectSize, objectSize + 16, objectSize))
zeroBin = np.zeros((projections, 2, nPixelsY))

inputs = []

# we have one dataset per y! with 10 channels (one for bone and one for water)
for i, bins in enumerate(y):
    # no that is not correct

    # input is 10 channels, zeroimage appended to the energy bin
    bin = bins.reshape((projections, nPixelsZ, nPixelsY))

    # normalize so biggest value is 1, smallest value is 0
    bin = (bin - np.min(bin)) / (np.max(bin) - np.min(bin))

    bin = np.concatenate((zeroBin, bin, zeroBin), axis=1)
    # for input image, concatenate bin to zeroImage in x direction
    input_image = np.concatenate((zeroImage, bin), axis=2)
    inputs.append(input_image)
    inputs.append(input_image)

return inputs

def prepareOutput(water, bone):

    water = np.flip(water, axis=1) # flip along z -axis
    water = np.flip(water, axis=2) # flip along y -axis
    bone = np.flip(bone, axis=1)   # flip along z -axis
    bone = np.flip(bone, axis=2)   # flip along y -axis

    zeroImage = np.zeros((32, 8, 32))
    zeroBin = np.zeros((projections, nPixelsZ + 4, nPixelsY)) # zero bin for the energy bin
    outputs = []

    bone_output = np.concatenate((zeroImage, bone, zeroImage), axis=1)
    water_output = np.concatenate((zeroImage, water, zeroImage), axis=1)

    bone_output = np.concatenate((bone_output, zeroBin), axis=2)
    water_output = np.concatenate((water_output, zeroBin), axis=2)

    outputs = [bone_output, water_output] # shape will be (10, 32, 44, 64)

    return outputs

# Prepare the input and output data
input = prepareInput(y)
output = prepareOutput(water, bone)

from ipywidgets import interact, IntSlider
import matplotlib.pyplot as plt

energy_bin = 0

```

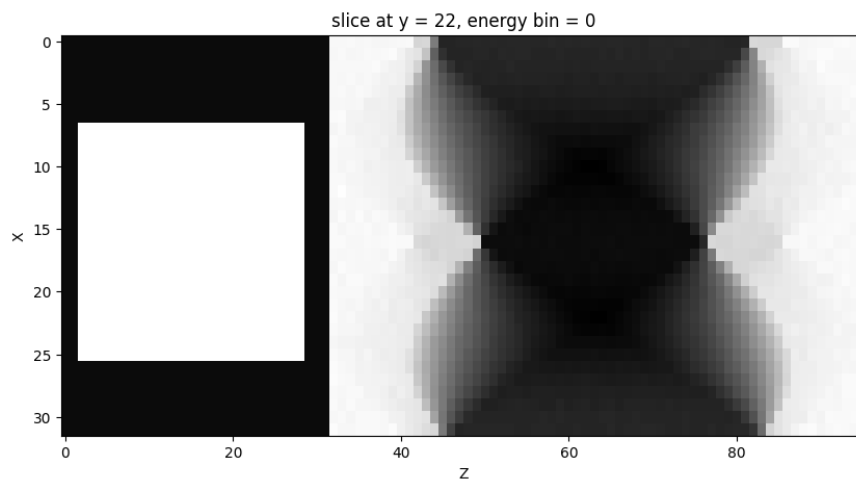
```
#invert the matrix, so z = 0 becomes z = max and y = 0 becomes y = max
flipped_bone_square = np.flip(bone, axis=1) # flip along z -axis
flipped_bone_square = np.flip(flipped_bone_square, axis=2) # flip along z -axis

#in input_square[0], replace[:, 8:40, :32] with the phantom
input[energy_bin][:, 8:40, :32] = flipped_bone_square[:, :, :]

# visualize with a slider, put slider for y value

plt.figure(figsize=(10, 5))
plt.imshow(input[energy_bin][:, 22, :], cmap='gray', aspect='auto')
plt.title(f'slice at y = 22, energy bin = {energy_bin}')
plt.xlabel('Z')
plt.ylabel('X')

# draw red rectangle around bottom and top padding
# draw red rectangle from (0, 0) to (32, 8) and (0, 40) to (32, 48)
plt.show()
```



4 Attention map

An important part of analysing the data is the attention mechanism, which allows the model to focus on different parts of the input data. Being able to visualise this mechanism is a key aspect of understanding how the network behaves. In this notebook I will

- Load preselected data
- Run the model on the preselected data
- Visualise the attention map on the input data

```
from libs.network.network import AttU_Net
import pickle
import os
import torch
import matplotlib.pyplot as plt
import torch.nn.functional as F
```

```
# Define paths
network_path = "data/network_att_map.pth"
data_path = "data/data_att_map.pkl"
```

```
# load torch input tensor
with open(data_path, "rb") as f:
    input_tensor = pickle.load(f)
```

```
torch.Size([8, 10, 32, 48, 96])
```

4.0.1 Load network and run output

```
network = AttU_Net()
network.load_state_dict(torch.load(network_path, map_location=torch.device('cpu'))) # Load the trained
network.eval()
```

```
output, attention_maps = network.getAttenuationMap(input_tensor)
```

4.1 Visualise attention mechanism

Define function `show_attention_overlay` which as an argument takes the input data, the attention weights, the slice axis and the slice index. For this example I have chosen to slice along the y-direction to properly visualise the sinogram.

```
def show_attention_overlay(input_tensor, attention, slice_axis=2, slice_idx=0, title="Attention Overlay")
    """
    Visualize attention over an input slice.

    Args:
        input_tensor: torch.Tensor [1, C, D, H, W]
        attention:     torch.Tensor [1, 1, D', H', W']
        slice_axis:    0=D, 1=H, 2=W (which axis to slice through)
        slice_idx:     index along that axis
    """
    assert input_tensor.ndim == 5 and attention.ndim == 5

    # Resize attention to match input spatial dims
```

```

attn_resized = F.interpolate(attention, size=input_tensor.shape[2:], mode='trilinear', align_corners=True)

# Select first input channel and remove batch dim
input_tensor = input_tensor[6, 0].detach().cpu()
attn_tensor = attn_resized[6, 0].detach().cpu()

# Normalize attention
attn_tensor = (attn_tensor - attn_tensor.min()) / (attn_tensor.max() - attn_tensor.min() + 1e-8)

# Choose slice
if slice_axis == 0: # Depth
    input_slice = input_tensor[slice_idx, :, :].numpy()
    attn_slice = attn_tensor[slice_idx, :, :].numpy()
elif slice_axis == 1: # Height
    input_slice = input_tensor[:, slice_idx, :].numpy()
    attn_slice = attn_tensor[:, slice_idx, :].numpy()
elif slice_axis == 2: # Width
    input_slice = input_tensor[:, :, slice_idx].numpy()
    attn_slice = attn_tensor[:, :, slice_idx].numpy()
else:
    raise ValueError("slice_axis must be 0 (D), 1 (H), or 2 (W)")

plt.figure(figsize=(10, 4))
# Plot overlay
plt.imshow(input_slice, cmap='gray')
plt.imshow(attn_slice, cmap='jet', alpha=0.3)
# make colourbar same height as image
plt.xlabel('Z')
plt.ylabel('X')
plt.colorbar(label='Attention coefficient', fraction=0.046, pad=0.04)
plt.title(title + f" (axis={slice_axis}, idx={slice_idx})")
plt.show()

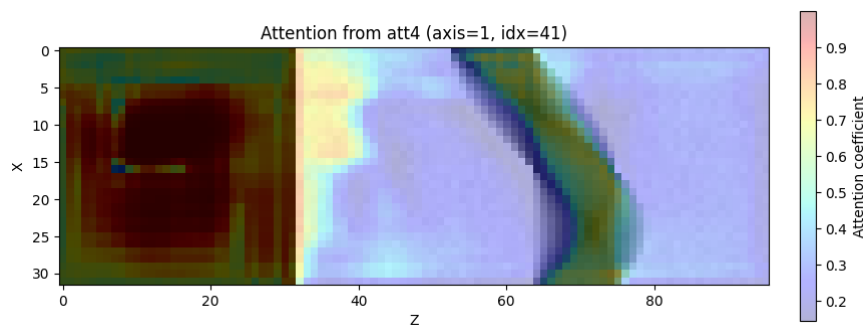
```

4.1.1 Plot attention map

```

attention = attention_maps['att2']
show_attention_overlay(input_tensor, attention, slice_axis=1, slice_idx=41, title="Attention from att4")

```



5 time-RMS plot

A time-rms plot which includes

- rms bone itt against time
- rms water itt against time
- rms bone model against time
- rms water model against time

```
import pickle
import matplotlib.pyplot as plt
import numpy as np

objectSize = 32

with open('phantom_results_2.pkl', 'rb') as f:
    data = pickle.load(f)

reconstructions = data['reconstructions']
ys = data['ys']
phantoms = data['phantoms']
model_images = data['output_images']
times = data['times']
times_model = data['times_images']
rms_itt_bone = data['rms_reconstructions_bone']
rms_itt_water = data['rms_reconstructions_water']
rms_model_bone = data['rms_models_bone']
rms_model_water = data['rms_models_water']

def rms_error(image1, image2):
    if image1.shape != image2.shape:
        raise ValueError("Images must have the same dimensions")

    # Calculate the squared differences
    squared_diff = (image1 - image2) ** 2

    # Calculate the mean of the squared differences
    mean_squared_diff = np.mean(squared_diff)

    # Return the square root of the mean squared difference
    return np.sqrt(mean_squared_diff)

# calculate RMS for reconstructed images

rms_reconstructions_bone = []
rms_reconstructions_water = []
rms_models_bone = []
rms_models_water = []

for reconstruction, output_image, phantom in zip(reconstructions, model_images, phantoms):

    rms_recon_bone = []
```

```

rms_recon_water = []
rms_model_bone = []
rms_model_water = []

phantom_bone = phantom[0].transpose() # Get the ith bone
phantom_water = phantom[1].transpose() # Get the ith water

# get first image from reconstruction
_, nMats, nIterates = reconstruction.shape
images = reconstruction.reshape((objectSize, objectSize, objectSize, nMats, nIterates), order = 'F')

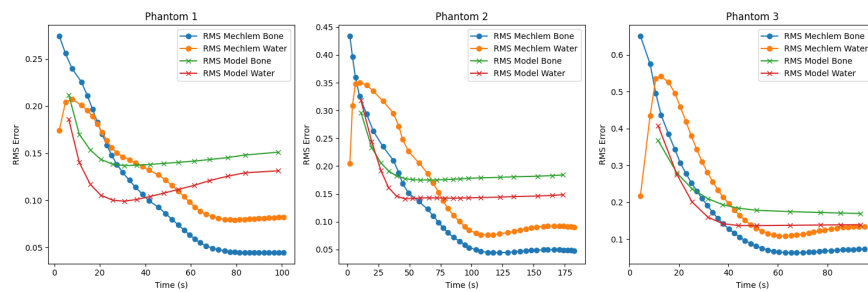
for i in range(len(images[0, 0, 0, 0, :])):
    image_bone = images[:, :, :, 0, i]
    image_water = images[:, :, :, 1, i]
    rms_recon_bone.append(rms_error(image_bone, phantom_bone))
    rms_recon_water.append(rms_error(image_water, phantom_water))
rms_reconstructions_bone.append(rms_recon_bone)
rms_reconstructions_water.append(rms_recon_water)

for image_bone, image_water in output_image:
    rms_model_bone.append(rms_error(image_bone, phantom_bone))
    rms_model_water.append(rms_error(image_water, phantom_water))
rms_models_bone.append(rms_model_bone)
rms_models_water.append(rms_model_water)

# we only want to keep the values for the models which are inside the time of the recon
for i in range(len(times_model)):
    times_model[i] = [t for t in times_model[i] if t <= max(times[i])]
# and the same for the RMS values
for i in range(len(rms_models_bone)):
    rms_models_bone[i] = rms_models_bone[i][:len(times_model[i])]
    rms_models_water[i] = rms_models_water[i][:len(times_model[i])]

# Plot the RMS errors for the reconstructions and the models
# make subplot with 1 row and len(reconstructions) columns
# plot them against the times
plt.figure(figsize=(15, 5))
for i in range(3):
    time_model = np.array(times_model[i])
    time_recon = np.array(times[i])
    plt.subplot(1, len(reconstructions), i + 1)
    plt.plot(time_recon, rms_reconstructions_bone[i], label='RMS Mechlem Bone', marker='o')
    plt.plot(time_recon, rms_reconstructions_water[i], label='RMS Mechlem Water', marker='o')
    plt.plot(time_model, rms_models_bone[i], label='RMS Model Bone', marker='x')
    plt.plot(time_model, rms_models_water[i], label='RMS Model Water', marker='x')
    plt.xlabel('Time (s)')
    plt.ylabel('RMS Error')
    plt.title(f'Phantom {i + 1}')
    plt.legend()
plt.tight_layout()
plt.show()

```



6 Figure displaying the final reconstructions

For the results we want to display 2 phantoms:

- 'good' reconstruction
- 'bad' reconstruction

We will use these reconstructions to point out the 'a model is only as good as it's data'

```
import pickle
import matplotlib.pyplot as plt
import numpy as np

objectSize = 32

with open('phantom_results_2.pkl', 'rb') as f:
    data = pickle.load(f)

reconstructions = data['reconstructions']
ys = data['ys']
phantoms = data['phantoms']
model_images = data['output_images']
times = data['times']
times_model = data['times_images']
rms_itt_bone = data['rms_reconstructions_bone']
rms_itt_water = data['rms_reconstructions_water']
rms_model_bone = data['rms_models_bone']
rms_model_water = data['rms_models_water']

def rms_error(image1, image2):
    if image1.shape != image2.shape:
        raise ValueError("Images must have the same dimensions")

    # Calculate the squared differences
    squared_diff = (image1 - image2) ** 2

    # Calculate the mean of the squared differences
    mean_squared_diff = np.mean(squared_diff)

    # Return the square root of the mean squared difference
    return np.sqrt(mean_squared_diff)

def get_best_image(reconstruction, bone, water):

    # calculate RMS error for each image, return best bone, water pair
    best_rms = float('inf')
    best_bone = None
    best_water = None

    _, nMats, nIterates = reconstruction.shape
    images = reconstruction.reshape((objectSize, objectSize, objectSize, nMats, nIterates), order = 'F')

    for i in range(nIterates):
```



```

bone_reconstructed = images[:, :, :, 0, i] # Maybe bone
water_reconstructed = images[:, :, :, 1, i] # Maybe water

# Calculate RMS error
rms_bone = rms_error(bone_reconstructed, bone)
rms_water = rms_error(water_reconstructed, water)
total_rms = rms_bone + rms_water
if total_rms < best_rms:
    best_rms = total_rms
    best_bone = bone_reconstructed
    best_water = water_reconstructed

return best_bone, best_water, best_rms

```

6.1 display reconstructions and GT

import interact, display for each index (as a slider) the final model_image, recon image and phantom
from ipywidgets import interact, FloatSlider, IntSlider, fixed

```
def display_images(index, itt):
```

```

    # get GT for water
    phantom = phantoms[index] # Get the ith phantom
    phantom_water = phantom[1].transpose() # Get the water phantom
    phantom_bone = phantom[0].transpose() # Get the bone phantom

    # get best iterative image
    reconstruction = reconstructions[index] # Get the ith reconstruction
    best_bone_image, best_water_image, rms = get_best_image(reconstruction, phantom_bone, phantom_water)

    # last iterative image
    _, nMats, nIterates = reconstruction.shape
    images = reconstruction.reshape((objectSize, objectSize, objectSize, nMats, nIterates), order = 'F')
    last_water_image = images[:, :, :, 1, -1] # Get the last water image
    last_bone_image = images[:, :, :, 0, -1] # Get the last bone image

    model_image = model_images[index][itt] # Get the ith output image
    model_bone = model_image[0] # Get the bone part of the model image
    model_water = model_image[1] # Get the water part of the model image

    # display ONLY water images
    plt.figure(figsize=(7, 4))
    plt.subplot(2, 4, 1)
    plt.imshow(phantom_water[:, :, objectSize//2], cmap='gray')
    plt.title(' (a) ')
    plt.axis('off')
    plt.subplot(2, 4, 2)
    plt.imshow(best_water_image[:, :, objectSize//2], cmap='gray')
    plt.title(f' (b) ')
    plt.axis('off')
    plt.subplot(2, 4, 3)
    plt.imshow(last_water_image[:, :, objectSize//2], cmap='gray')
    plt.title(' (c) ')
    plt.axis('off')

```

```

plt.subplot(2, 4, 4)
plt.imshow(model_water[:, :, objectSize//2], cmap='gray')
plt.title(f' (d) ')
plt.axis('off')
plt.subplot(2, 4, 5)
plt.imshow(phantom_bone[:, :, objectSize//2], cmap='gray')
plt.title(' (e) ')
plt.axis('off')
plt.subplot(2, 4, 6)
plt.imshow(best_bone_image[:, :, objectSize//2], cmap='gray')
plt.title(f' (f) ')
plt.axis('off')
plt.subplot(2, 4, 7)
plt.imshow(last_bone_image[:, :, objectSize//2], cmap='gray')
plt.title(' (g) ')
plt.axis('off')
plt.subplot(2, 4, 8)
plt.imshow(model_bone[:, :, objectSize//2], cmap='gray')
plt.title(f' (h) ')
plt.axis('off')

```

```

interact(display_images, index=IntSlider(min=0, max=len(model_images) -1, step=1, value=0, description=
interactive(children=(IntSlider(value=0, description='Index', max=2), IntSlider(value=0, description='I
<function __main__.display_images(index, itt)>

```

6.2 Image to display

choose to display phantom 1 at iteration 3

```

index = 1
itt = 3

# get GT for water
phantom = phantoms[index] # Get the ith phantom
phantom_water = phantom[1].transpose() # Get the water phantom
phantom_bone = phantom[0].transpose() # Get the bone phantom

# get best iterative image
reconstruction = reconstructions[index] # Get the ith reconstruction
best_bone_image , best_water_image, rms = get_best_image(reconstruction, phantom_bone, phantom_water)

# last iterative image
_, nMats, nIterates = reconstruction.shape
images = reconstruction.reshape((objectSize, objectSize, objectSize, nMats, nIterates), order = 'F')
last_water_image = images[:, :, :, 1, -1] # Get the last water image
last_bone_image = images[:, :, :, 0, -1] # Get the last bone image

model_image = model_images[index][itt] # Get the ith output image
model_bone = model_image[0] # Get the bone part of the model imag

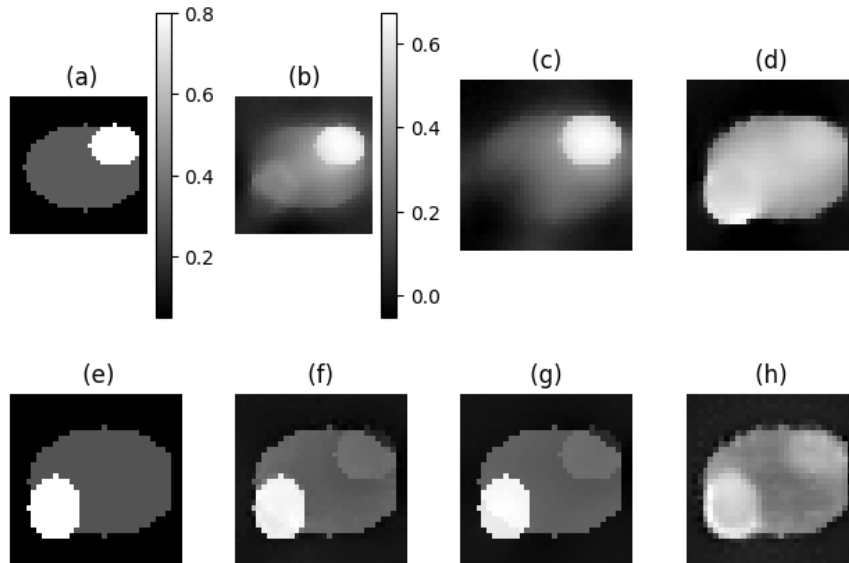
```

```

model_water = model_image[1] # Get the water part of the model image

# display ONLY water images
# get constant colorbar for all images
plt.subplot(2, 4, 1)
plt.imshow(phantom_water[:, :, objectSize//2], cmap='gray')
plt.title(' (a) ')
plt.axis('off')
plt.subplot(2, 4, 2)
plt.imshow(best_water_image[:, :, objectSize//2], cmap='gray')
plt.title(f' (b) ')
plt.axis('off')
plt.subplot(2, 4, 3)
plt.imshow(last_water_image[:, :, objectSize//2], cmap='gray')
plt.title(' (c) ')
plt.axis('off')
plt.subplot(2, 4, 4)
plt.imshow(model_water[:, :, objectSize//2], cmap='gray')
plt.title(f' (d) ')
plt.axis('off')
plt.subplot(2, 4, 5)
plt.imshow(phantom_bone[:, :, objectSize//2], cmap='gray')
plt.title(' (e) ')
plt.axis('off')
plt.subplot(2, 4, 6)
plt.imshow(best_bone_image[:, :, objectSize//2], cmap='gray')
plt.title(f' (f) ')
plt.axis('off')
plt.subplot(2, 4, 7)
plt.imshow(last_bone_image[:, :, objectSize//2], cmap='gray')
plt.title(' (g) ')
plt.axis('off')
plt.subplot(2, 4, 8)
plt.imshow(model_bone[:, :, objectSize//2], cmap='gray')
plt.title(f' (h) ')
plt.axis('off')
plt.tight_layout()
plt.show()

```



```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(12, 6))
gs = gridspec.GridSpec(2, 5, width_ratios=[1, 1, 1, 1, 0.05], wspace=0.3)

# First row
ax1 = fig.add_subplot(gs[0, 0])
im = ax1.imshow(phantom_water[:, :, objectSize//2], cmap='gray')
ax1.set_title(' (a) ')
ax1.axis('off')

ax2 = fig.add_subplot(gs[0, 1])
ax2.imshow(best_water_image[:, :, objectSize//2], cmap='gray')
ax2.set_title(' (b) ')
ax2.axis('off')

ax3 = fig.add_subplot(gs[0, 2])
ax3.imshow(last_water_image[:, :, objectSize//2], cmap='gray')
ax3.set_title(' (c) ')
ax3.axis('off')

ax4 = fig.add_subplot(gs[0, 3])
ax4.imshow(model_water[:, :, objectSize//2], cmap='gray')
ax4.set_title(' (d) ')
ax4.axis('off')

# Second row
ax5 = fig.add_subplot(gs[1, 0])
ax5.imshow(phantom_bone[:, :, objectSize//2], cmap='gray')
ax5.set_title(' (e) ')
ax5.axis('off')

ax6 = fig.add_subplot(gs[1, 1])
ax6.imshow(best_bone_image[:, :, objectSize//2], cmap='gray')
```

```

ax6.set_title(' (f)')
ax6.axis('off')

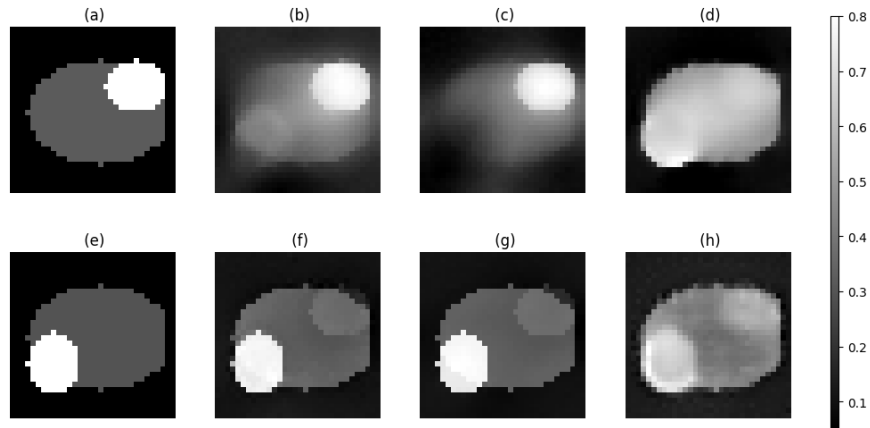
ax7 = fig.add_subplot(gs[1, 2])
ax7.imshow(last_bone_image[:, :, objectSize//2], cmap='gray')
ax7.set_title(' (g)')
ax7.axis('off')

ax8 = fig.add_subplot(gs[1, 3])
ax8.imshow(model_bone[:, :, objectSize//2], cmap='gray')
ax8.set_title(' (h)')
ax8.axis('off')

# Add colorbar on dedicated axis (right column)
cax = fig.add_subplot(gs[:, 4])
fig.colorbar(im, cax=cax)

# fig.tight_layout()
plt.show()

```



```

import pickle
import numpy as np
import matplotlib.pyplot as plt

## SETTINGS ##
objectSize = 32                                # Object Size a square
nPixelsY = 64                                  # Number of pixels on the detector plate.
nPixelsZ = 44                                  # Number of pixels on the detector plate.
pixelPitch = 1                                 # Pixel pitch
nPixelsPerProj = nPixelsZ * nPixelsY           # The total number of pixels on the detector plate
projections = 32                               # Number of projections
nSubset = 1
nIter = 40

data_file = "phantom_results_3.pkl"

def rms_error(image1, image2):
    if image1.shape != image2.shape:
        raise ValueError("Images must have the same dimensions")

    # Calculate the squared differences
    squared_diff = (image1 - image2) ** 2

    # Calculate the mean of the squared differences
    mean_squared_diff = np.mean(squared_diff)

    # Return the square root of the mean squared difference
    return np.sqrt(mean_squared_diff)

# import pickle data
with open(data_file, 'rb') as f:
    data = pickle.load(f)

print(data.keys())
ys = data['ys']
times = data['times']
reconstructions = data['reconstructions']
phantoms = data['phantoms']
model_images = data['output_images']
model_times = data['times_images']

dict_keys(['reconstructions', 'ys', 'times', 'phantoms', 'output_images', 'times_images', 'rms_reconstr

# calculate RMS for reconstructed images

rms_reconstructions_bone = []
rms_reconstructions_water = []
rms_models_bone = []
rms_models_water = []

for reconstruction, output_image, phantom in zip(reconstructions, model_images, phantoms):

    rms_recon_bone = []
    rms_recon_water = []
    rms_model_bone = []

```

```

rms_model_water = []

phantom_bone = phantom[0].transpose() # Get the ith bone
phantom_water = phantom[1].transpose() # Get the ith water

# get first image from reconstruction
_, nMats, nIterates = reconstruction.shape
images = reconstruction.reshape((objectSize, objectSize, objectSize, nMats, nIterates), order = 'F')

for i in range(len(images[0, 0, 0, 0, :])):
    image_bone = images[:, :, :, 0, i]
    image_water = images[:, :, :, 1, i]
    rms_recon_bone.append(rms_error(image_bone, phantom_bone))
    rms_recon_water.append(rms_error(image_water, phantom_water))
rms_reconstructions_bone.append(rms_recon_bone)
rms_reconstructions_water.append(rms_recon_water)

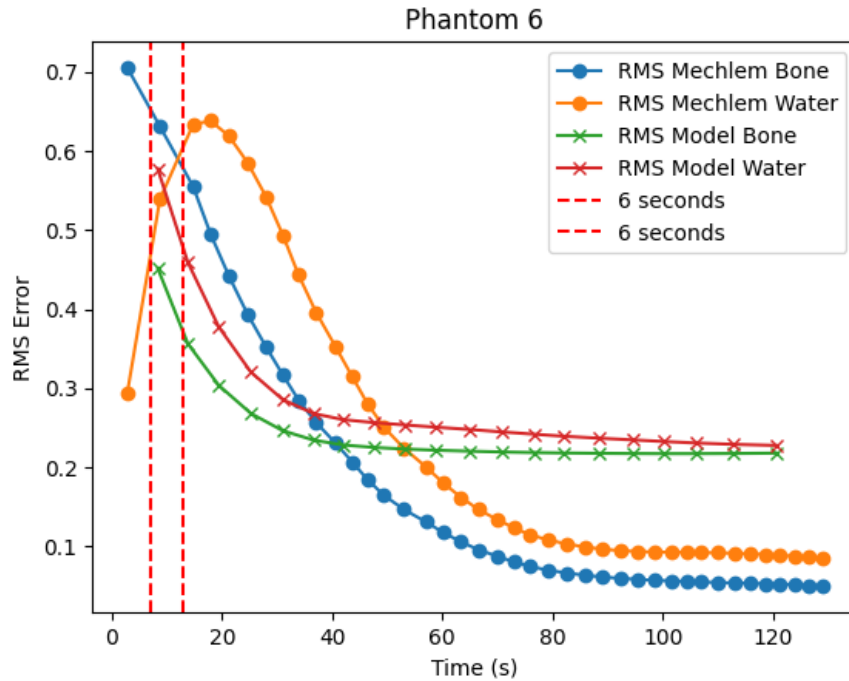
for image_bone, image_water in output_image:
    rms_model_bone.append(rms_error(image_bone, phantom_bone))
    rms_model_water.append(rms_error(image_water, phantom_water))
rms_models_bone.append(rms_model_bone)
rms_models_water.append(rms_model_water)

i = 5
time_model = np.array(model_times[i])
time_recon = np.array(times[i])

plt.plot(time_recon, rms_reconstructions_bone[i], label='RMS Mechlem Bone', marker='o')
plt.plot(time_recon, rms_reconstructions_water[i], label='RMS Mechlem Water', marker='o')
plt.plot(time_model, rms_models_bone[i], label='RMS Model Bone', marker='x')
plt.plot(time_model, rms_models_water[i], label='RMS Model Water', marker='x')
# vertical bar at 6
plt.axvline(x=7, color='r', linestyle='--', label='6 seconds')
plt.axvline(x=12.9, color='r', linestyle='--', label='6 seconds')

plt.xlabel('Time (s)')
plt.ylabel('RMS Error')
plt.title(f'Phantom {i + 1}')
plt.legend()
plt.show()

```



```
first_times = [t[0] for t in model_times]
print("first times:", first_times)
print("average first time:", np.mean(first_times), "+ -", np.std(first_times), "seconds")
```

```
speedups = []
interpolated_speedups = []
```

```
for i in range(len(rms_reconstructions_bone)):
    rms_bone_model = rms_models_bone[i][0]
    rms_water_model = rms_models_water[i][0]

    for j in range(len(rms_reconstructions_bone[i])):

        if(rms_reconstructions_bone[i][j] < rms_bone_model):

            speedup = times[i][j] / first_times[i]
            speedups.append(speedup)

            # also interpolate to see at what time the rms is the same
            slope = (rms_reconstructions_bone[i][j+1] - rms_reconstructions_bone[i][j]) / (times[i][j+1] - times[i][j])

            # see for what time the rms is the same
            if slope != 0:
                interpolated_time = (rms_bone_model - rms_reconstructions_bone[i][j]) / slope + times[i][j]
                interpolated_speedups.append(interpolated_time / first_times[i])
            else:
                interpolated_speedups.append(speedup)
            break
```

```
print("average speedup:", np.mean(speedups), "+ -", np.std(speedups), "after", np.mean(first_times), "+ -", np.std(first_times), "seconds")
```



```

print("average interpolated speedup:", np.mean(interpolated_speedups), "+ -", np.std(interpolated_speedups))

first times: [6.995673656463623, 8.126861333847046, 9.385169744491577, 13.482990264892578, 8.5782411098]
average first time: 9.347357761859893 + - 1.5048523200384376 seconds
average speedup: 1.6119690273535912 + - 0.3327647687994946 after 9.347357761859893 + - 1.5048523200384376 seconds
average interpolated speedup: 1.4428355769342633 + - 0.3332045100786529 after 9.347357761859893 + - 1.5048523200384376 seconds

itt = 5

time_model = model_times[itt]
time_recon = times[itt]

rms_model_bone = rms_models_bone[itt]
rms_model_water = rms_models_water[itt]
rms_recon_bone = rms_reconstructions_bone[itt]
rms_recon_water = rms_reconstructions_water[itt]

interpolated_speedup = []

for idx in range(len(rms_model_bone)):

    rms_bone = rms_model_bone[idx]
    rms_water = rms_model_water[idx]

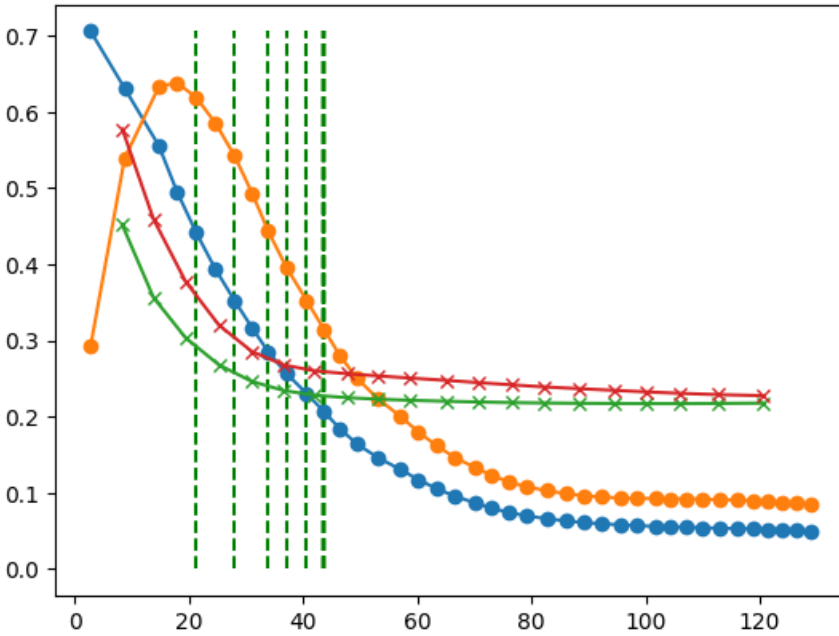
    # print(f"RMS Model Bone: {rms_bone:.4f}, RMS Model Water: {rms_water:.4f}")
    for i in range(len(rms_recon_bone)):
        if(rms_recon_bone[i] < rms_bone):
            plt.vlines(x=time_recon[i], ymin=0, ymax=max(rms_recon_bone + rms_recon_water), color='g', linestyle='solid')

            # interpolate to get speedup
            slope = (rms_recon_bone[i+1] - rms_recon_bone[i]) / (time_recon[i+1] - time_recon[i])
            if slope != 0:
                interpolated_time = (rms_bone - rms_recon_bone[i]) / slope + time_recon[i]
                interpolated_speedup.append(interpolated_time / time_model[idx])
            else:
                interpolated_speedup.append(time_recon[i] / time_model[idx])
            break
        pass

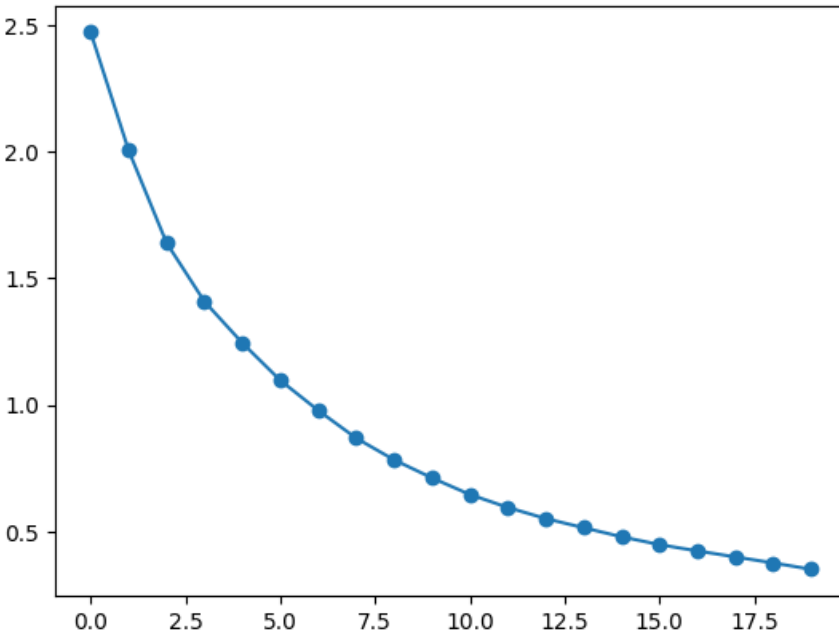
plt.plot(time_recon, rms_recon_bone, label='RMS Mechlem Bone', marker='o')
plt.plot(time_recon, rms_recon_water, label='RMS Mechlem Water', marker='o')
plt.plot(time_model, rms_model_bone, label='RMS Model Bone', marker='x')
plt.plot(time_model, rms_model_water, label='RMS Model Water', marker='x')
plt.show()

plt.plot(interpolated_speedup, label='Interpolated Speedup', marker='o')

```



[<matplotlib.lines.Line2D at 0x254a50e0580>]



```
all_interpolated_speedups_bone = [] # List of lists - one list per phantom
all_interpolated_speedups_water = [] # List of lists - one list per phantom
```

```
for itt in range(len(times)): # Loop over phantoms
    time_model = model_times[itt]
    time_recon = times[itt]
```

```
    rms_model_bone = rms_models_bone[itt]
    rms_model_water = rms_models_water[itt]
```

```

rms_recon_bone = rms_reconstructions_bone[itt]
rms_recon_water = rms_reconstructions_water[itt]

interpolated_speedup_bone = []
interpolated_speedup_water = []

for idx in range(len(rms_model_bone)): # Loop over model iterations (idx)
    rms_bone = rms_model_bone[idx]
    rms_water = rms_model_water[idx]

    for i in range(len(rms_recon_bone) - 1): # Loop over recon steps
        if rms_recon_bone[i] < rms_bone:
            # interpolate to get speedup
            slope = (rms_recon_bone[i+1] - rms_recon_bone[i]) / (time_recon[i+1] - time_recon[i])
            if slope != 0:
                interpolated_time = (rms_bone - rms_recon_bone[i]) / slope + time_recon[i]
                interpolated_speedup_bone.append(interpolated_time / time_model[idx])
            else:
                interpolated_speedup_bone.append(time_recon[i] / time_model[idx])
            break # Once found, move to next model iteration

    for i in range(len(rms_recon_water) - 1): # Loop over recon steps for water
        if rms_recon_water[i] < rms_water:
            # interpolate to get speedup
            slope = (rms_recon_water[i+1] - rms_recon_water[i]) / (time_recon[i+1] - time_recon[i])
            if slope != 0:
                interpolated_time = (rms_water - rms_recon_water[i]) / slope + time_recon[i]
                interpolated_speedup_water.append(interpolated_time / time_model[idx])
            else:
                interpolated_speedup_water.append(time_recon[i] / time_model[idx])
            break

    all_interpolated_speedups_water.append(interpolated_speedup_water) # Save speedup curve for this p
    all_interpolated_speedups_bone.append(interpolated_speedup_bone)

max_len_bone = max(len(x) for x in all_interpolated_speedups_bone)
max_len_water = max(len(x) for x in all_interpolated_speedups_water)

bone_array = np.array([x + [np.nan] * (max_len_bone - len(x)) for x in all_interpolated_speedups_bone])
water_array = np.array([x + [np.nan] * (max_len_water - len(x)) for x in all_interpolated_speedups_water])

mean_speedups_bone = np.nanmean(bone_array, axis=0)
std_speedups_bone = np.nanstd(bone_array, axis=0)

mean_speedups_water = np.nanmean(water_array, axis=0)
std_speedups_water = np.nanstd(water_array, axis=0)

plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.errorbar(range(len(mean_speedups_bone)), mean_speedups_bone, yerr
=std_speedups_bone, fmt='o', label='Bone Speedup', capsizes=5)
plt.xlabel('Iteration')
plt.ylabel('Mean Speedup')
plt.title('( a )')

```

```

plt.xticks(range(0, len(mean_speedups_bone), 2))
plt.legend()
plt.subplot(1, 2, 2)
plt.errorbar(range(len(mean_speedups_water)), mean_speedups_water, yerr=std_speedups_water, fmt='o', label='Water Speedup')
plt.xlabel('Iteration')
plt.ylabel('Mean Speedup')
plt.title('( b )')
plt.legend()
plt.xticks(range(0, len(mean_speedups_bone), 2))
plt.tight_layout()
plt.show()

```

